customer expects. TDD also does not replace some kind of quality-assurance phase where users or testers pound away at the actual program trying to break something. Quality-assurance testing can expose flaws in the developer's understanding of the task that are unlikely to be exposed by the developer's own tests.

Further, TDD does not replace the role of a traditional software tester. TDD is simply a development process that produces better and more accurate code. A separate verification phase run by people who are not the original developers is still a good idea. For a thorough overview of more traditional exploratory testing, read *Explore It! Reduce Risk and Increase Confidence with Exploratory Testing [Hen13]*.

Verification is valuable, but the idea of verification can be taken too far. You sometimes see an argument against test-driven development that says, "The purpose of testing is to verify that my program is correct. I can never prove correctness with 100 percent certainty. Therefore, testing has no value." (RSpec, which is designed for a process its creators called behavior-driven development, was created to combat this attitude.) Ultimately, though, testing has a lot of benefits to offer for coding, even beyond verification.

Preventing regression is often presented as one of the paramount benefits of a test-driven development process. And if you're expecting me to disagree out of spite, you're out of luck. Being able to squash regressions before anybody else sees them is one of the key ways in which strict testing will speed up your development over time.

You may have heard that automated tests provide an alternative method of documenting your program—that the tests provide a detailed functional specification of the program's behavior. That's the theory. My experience with tests acting as documentation is mixed, to say the least. Still, it's useful to keep this in mind as a goal. Tests can be valuable in documenting the expected behavior of the system when things go wrong, and most of the things that make tests work better as documentation will also make the tests work better, period.

To make your tests effective as documentation, focus on giving them names that describe the reason for their existence, and keeping tests short and simple. When a test has a lot of complex setup and assertions, it can be hard for a reader to focus on the important features. As you'll see, a test that requires a bunch of tricky setup often indicates a problem in the underlying code.

In a testing environment, blank-page problems are almost completely nonexistent. I can always think of something that the program needs to do, so I write a test for that. When you're working test-first, the order in which pieces are